Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 20
NP-Completeness (Part 1)
NP-Πληρότητα (Μέρος 1)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

Problems, Instances, and Solutions

- A **problem** (**πρόβλημα**) is a general question to be answered, usually possessing several **parameters** (**παραμέτρους**), whose values are left *unspecified*
- An **instance** (**στιγμιότυπο**) of a problem is obtained by specifying particular values for all the problem parameters
- An algorithm is said to **solve** (**επιλύει**) a problem P if that algorithm can be applied to *any* instance I of P and is guaranteed always to produce a solution for that instance I

Tractability

- Some problems are **intractable** (**δυσεπίλυτα**): as they grow large, we are unable to solve them in reasonable time (εύλογο χρονικό διάστημα)
- What constitutes reasonable time?

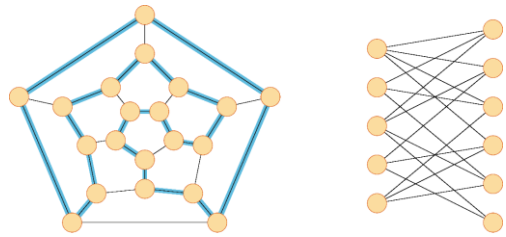
  Standard working definition: **polynomial time** (**πολυωνυμικό χρόνο**)
  - On an input of size n, the worst-case running time is $O(n^k)$ for some constant k
  - Polynomial time: $O(n^2), O(n^3), O(1), O(n \lg n)$
  - Not in polynomial time: $O(2^n), O(n^n), O(n!)$

Polynomial-Time Algorithms

- Are some problems solvable in polynomial time?
  - *Of course*: almost all algorithms we've studied provide polynomial-time solutions to some problem
  - We define **P** to be the class of problems solvable in polynomial time
- Are all problems solvable in polynomial time?
  - *No*: Turing's "Halting Problem" is not solvable by any computer, no matter how much time is given
  - Such problems are clearly intractable, not in **P**

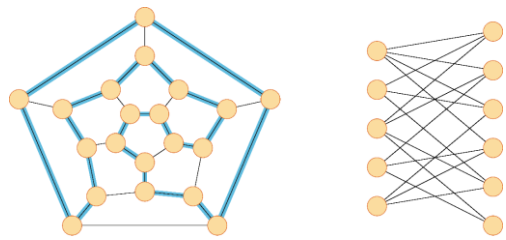
Hamiltonian Cycles

- A **simple cycle** (**απλός κύκλος**) in a graph $G = (V, E)$ is a sequence $\langle v_1, v_2, \ldots, v_k \rangle$ of distinct vertices from V such that $(v_i, v_{i+1}) \in E$ for all $i, 1 \leq i \leq k$ and $(v_k, v_1) \in E$
- A **hamiltonian cycle** of an undirected graph is a simple cycle that contains every vertex
- The **hamiltonian-cycle problem**: Given a graph G, does it have a hamiltonian cycle?



Hamiltonian Cycles

- The **hamiltonian-cycle problem**: Given a graph G, does it have a hamiltonian cycle?
  - There is **no** known algorithm that can solve this problem in polynomial time
  - Describe a naïve algorithm for solving the hamiltonian-cycle problem. *Running time?*



P and NP

- As mentioned, **P** is the set of problems that can be solved in polynomial time
- **NP** (**nondeterministic polynomial time**) is the set of problems that are **verifiable** (**επαληθεύσιμα**) in polynomial time
  - Given a solution, we can verify whether it is correct or not in polynomial time
- Hamiltonian-cycle problem is in **NP**:
  - Cannot solve in polynomial time
  - Easy to verify solution in polynomial time (*How?*)
- Any problem in **P** is also in **NP** (*Why?*)
- Unknown whether **P** = **NP** (most suspect not)

NP-Complete Problems

- The **NP-Complete** problems are an interesting class of problems whose status is unknown
  - No polynomial-time algorithm has been discovered for an NP-Complete problem
  - No proof that a polynomial-time algorithm cannot exist for any of the NP-Complete problems

NP-Complete Problems

- We will see that **NP-Complete** problems are the "hardest" problems in **NP**:
  - If any *one* NP-Complete problem can be solved in polynomial time…
  - …then *every* NP-Complete problem can be solved in polynomial time…
  - …and in fact *every* problem in **NP** can be solved in polynomial time (which would show **P = NP**)
  - *Thus*: solve hamiltonian cycle problem in $O(n^{100})$ time and you will prove that **P = NP**. Retire rich & famous!

Why Discuss NP-Complete Problems?

- If a problem is proved to be NPC, this is a good evidence for its intractability (hardness)
- Do not waste time on trying to find efficient algorithm for it
- Instead, focus on designing an approximate algorithm or a solution for a special case of the problem
- Some problems looks very easy on the surface, but in fact, are hard (NPC)
  - Longest simple path
  - Hamiltonian cycle
  - 3-CNF satisfiability
  - …

Optimization Problems vs Decision Problems

- An **optimization problem** (**πρόβλημα βελτιστοποίησης**) aims to optimize some value, e.g., find a minimum "cost"
  - Example: **SHORTEST-PATH**
    Given G, u, v, find a path from u to v in G with fewest edges
- A **decision problem** (**πρόβλημα απόφασης**) is a problem that has only two possible solutions, either the answer "yes" or the answer "no"
  - Example: **PATH**
    Given G, u, v, k, does a path exist from u to v in G consisting of at most k edges?

Optimization Problems vs Decision Problems

- A decision problem is **no harder** than corresponding optimization problem
  - If there is an algorithm for an optimization problem, the algorithm can be used to solve the corresponding decision problem
  - Example: Solve PATH using SHORTEST-PATH (*How?*)
- If optimization is easy, its corresponding decision is also easy
- If decision is hard, its corresponding optimization is also hard
- Thus, the theory of NP-completeness restricts attention to *only* decision problems
  - We can extend the implications of the theory to optimization problems as well
  - It's easy to define **reduction (αναγωγή)** between decision problems

(Poly) Reduction between Decision Problems

- Suppose we want to solve a decision problem A
- We know how to solve a different decision problem B in polynomial time
- A **reduction** (**αναγωγή**) from A to B is a transformation with the following properties:
  - The transformation takes polynomial time
  - The answer is the same (the answer for an instance of A is YES if and only if the answer for an instance of B is YES)

(Poly) Reduction between Decision Problems

Example:

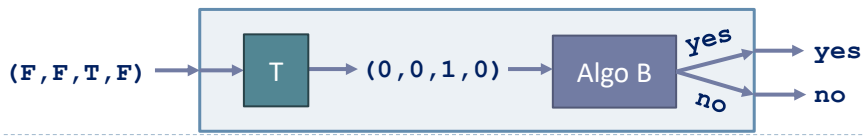
- A: Given a list of booleans, is at least one TRUE?



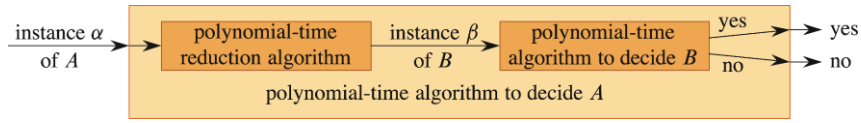
- B: Given a list of integers, is their sum positive (> 0)?



- Transformation: $(x_1, x_2, \ldots, x_n) \rightarrow (y_1, y_2, \ldots, y_n)$
  where $y_i = 1$ if $x_i = \text{TRUE}$ and $y_i = 0$ if $x_i = \text{FALSE}$



Implication of (Poly) Reduction



- If decision algorithm for B is polynomial, so is A
  - A is no harder than B (or B is no easier than A)
- Consider the reverse situation:
  - Suppose we know A is hard (e.g., NPC)
  - If we can reduce A to B, then B is hard as well
- **How to prove a problem B to be NPC?**
  - Prove B is in NP (which is generally easy)
  - Find an already proved NPC problem A
  - Establish a (poly) reduction from A to B